Ejemplos de programación funcional

Tres teorías de ejemplos de la Parte II

Razonamiento Automático — Isabelle/HOL

Contenido

Listas: `EjemplosTipoListas.thy`

- El tipo de datos lista
- Tres estilos para la misma demostración
- Inducción sobre listas

Contenido

Listas: `EjemplosTipoListas.thy`

- El tipo de datos lista
- Tres estilos para la misma demostración
- Inducción sobre listas

Naturales: `EjemplosTipoNumerosNaturales.thy`

- Los números naturales
- Dos maneras de decir "par"
- Inducción estructural

Contenido

Listas: `EjemplosTipoListas.thy`

- El tipo de datos lista
- Tres estilos para la misma demostración
- Inducción sobre listas

Naturales: `EjemplosTipoNumerosNaturales.thy`

- Los números naturales
- Dos maneras de decir "par"
- Inducción estructural

Más listas: `OtrosEjemplosListas.thy`

- Recursión sobre dos argumentos
- Cuantificar sobre una lista

Contenido

Listas: `EjemplosTipoListas.thy`

- El tipo de datos lista
- Tres estilos para la misma demostración
- Inducción sobre listas

Naturales: `EjemplosTipoNumerosNaturales.thy`

- Los números naturales
- Dos maneras de decir "par"
- Inducción estructural

Más listas: `OtrosEjemplosListas.thy`

- Recursión sobre dos argumentos
- Cuantificar sobre una lista

Cierre

`EjemplosTipoListas.thy`	listas
`EjemplosTipoNumerosNaturales.thy`	números naturales

`EjemplosTipoListas.thy`	listas
`EjemplosTipoNumerosNaturales.thy`	números naturales
`OtrosEjemplosListas.thy`	más listas

`EjemplosTipoListas.thy`	listas
`EjemplosTipoNumerosNaturales.thy`	números naturales
`OtrosEjemplosListas.thy`	más listas

- Definimos funciones sobre listas y sobre naturales.

`EjemplosTipoListas.thy`	listas
`EjemplosTipoNumerosNaturales.thy`	números naturales
`OtrosEjemplosListas.thy`	más listas

- Definimos funciones sobre listas y sobre naturales.
- Las evaluamos con `value`, como en cualquier lenguaje funcional.

`EjemplosTipoListas.thy`	listas
`EjemplosTipoNumerosNaturales.thy`	números naturales
`OtrosEjemplosListas.thy`	más listas

- Definimos funciones sobre listas y sobre naturales.
- Las evaluamos con `value`, como en cualquier lenguaje funcional.
- Y demostramos sus propiedades por inducción, con el mismo lenguaje de demostración de la Parte I.

`EjemplosTipoListas.thy`	listas
`EjemplosTipoNumerosNaturales.thy`	números naturales
`OtrosEjemplosListas.thy`	más listas

- Definimos funciones sobre listas y sobre naturales.
- Las evaluamos con `value`, como en cualquier lenguaje funcional.
- Y demostramos sus propiedades por inducción, con el mismo lenguaje de demostración de la Parte I.

Abra cada teoría en jEdit cuando lleguemos a ella y vaya siguiendo los ejemplos.

Listas: `EjemplosTipoListas.thy`

Las listas son un tipo recursivo

La librería de Isabelle define:

```
datatype 'a list = Nil    ("[]")
                 | Cons 'a "'a list"  (infixr "#" 65)
```

Las listas son un tipo recursivo

La librería de Isabelle define:

```
datatype 'a list = Nil    ("[]")
                 | Cons 'a "'a list"  (infixr "#" 65)
```

- `Nil`: la lista vacía.

Las listas son un tipo recursivo

La librería de Isabelle define:

```
datatype 'a list = Nil    ("[]")
                 | Cons 'a "'a list"  (infixr "#" 65)
```

- `Nil`: la lista vacía.
- `Cons x xs`: añade `x` al frente de `xs`.

Las listas son un tipo recursivo

La librería de Isabelle define:

```
datatype 'a list = Nil    ("[]")
                 | Cons 'a "'a list"  (infixr "#" 65)
```

- `Nil`: la lista vacía.
- `Cons x xs`: añade `x` al frente de `xs`.
- Lo que va entre paréntesis es solo notación: `Nil` se escribe `[]`, y `Cons x xs` se escribe `x # xs`.

Las listas son un tipo recursivo

La librería de Isabelle define:

```
datatype 'a list = Nil    ("[]")
                 | Cons 'a "'a list"  (infixr "#" 65)
```

- `Nil`: la lista vacía.
- `Cons x xs`: añade `x` al frente de `xs`.
- Lo que va entre paréntesis es solo notación: `Nil` se escribe `[]`, y `Cons x xs` se escribe `x # xs`.

Toda lista es `[]` o `x # xs`. No hay un tercer caso: de ahí saldrán las demostraciones.

Tres maneras de escribir la misma lista

```
value "Cons (3::nat) Nil"                    ⇝ [3]
value "((3::nat) # Nil)"                     ⇝ [3]
```

Tres maneras de escribir la misma lista

```
value "Cons (3::nat) Nil"                                    ⇝ [3]
value "((3::nat) # Nil)"                                     ⇝ [3]

lemma "Cons (3::nat) Nil == ((3::nat) # Nil)" .
lemma "Cons (3::nat) Nil == [3]" .
lemma "2 # Cons 3 Nil == [2,3]" .
lemma "1 # 2 # 3 # Nil == [1, 2, 3]" .
```

Tres maneras de escribir la misma lista

```
value "Cons (3::nat) Nil"                                    ⇝ [3]
value "((3::nat) # Nil)"                                     ⇝ [3]

lemma "Cons (3::nat) Nil == ((3::nat) # Nil)" .
lemma "Cons (3::nat) Nil == [3]" .
lemma "2 # Cons 3 Nil == [2,3]" .
lemma "1 # 2 # 3 # Nil == [1, 2, 3]" .
```

Las cuatro demostraciones son un punto.

¿Por qué basta un punto?

- == es $\equiv$, la igualdad de definición de Isabelle.

¿Por qué basta un punto?

- == es $\equiv$, la igualdad de definición de Isabelle.
- `[3]`, `3 # Nil` y `Cons 3 Nil` no son tres cosas parecidas: son la misma expresión escrita de tres maneras.

¿Por qué basta un punto?

- == es $\equiv$, la igualdad de definición de Isabelle.
- `[3]`, `3 # Nil` y `Cons 3 Nil` no son tres cosas parecidas: son la misma expresión escrita de tres maneras.
- Isabelle deshace la notación antes de mirar nada, y los dos lados quedan idénticos.

¿Por qué basta un punto?

- == es $\equiv$, la igualdad de definición de Isabelle.
- `[3]`, `3 # Nil` y `Cons 3 Nil` no son tres cosas parecidas: son la misma expresión escrita de tres maneras.
- Isabelle deshace la notación antes de mirar nada, y los dos lados quedan idénticos.

El punto, en la Parte I y aquí

. significa "esto ya está": se cierra con lo que hay a la mano, sin hacer ningún trabajo.

Concatenación: la de la librería

En la teoría de listas de Isabelle:

```
primrec append :: "'a list ⇒ 'a list ⇒ 'a list"  (infixr "@" 65) where
  append_Nil:  "[] @ ys = ys"
| append_Cons: "(x#xs) @ ys = x # (xs @ ys)"
```

Concatenación: la de la librería

En la teoría de listas de Isabelle:

```
primrec append :: "'a list ⇒ 'a list ⇒ 'a list"  (infixr "@" 65) where
  append_Nil:  "[] @ ys = ys"
| append_Cons: "(x#xs) @ ys = x # (xs @ ys)"
```

- Dos ecuaciones: una por constructor del primer argumento.

Concatenación: la de la librería

En la teoría de listas de Isabelle:

```
primrec append :: "'a list ⇒ 'a list ⇒ 'a list"  (infixr "@" 65) where
  append_Nil:  "[] @ ys = ys"
| append_Cons: "(x#xs) @ ys = x # (xs @ ys)"
```

- Dos ecuaciones: una por constructor del primer argumento.
- La recursión va sobre la primera lista; la segunda viaja intacta.

Concatenación: la de la librería

En la teoría de listas de Isabelle:

```
primrec append :: "'a list ⇒ 'a list ⇒ 'a list"  (infixr "@" 65) where
  append_Nil:  "[] @ ys = ys"
| append_Cons: "(x#xs) @ ys = x # (xs @ ys)"
```

- Dos ecuaciones: una por constructor del primer argumento.
- La recursión va sobre la primera lista; la segunda viaja intacta.
- Cada ecuación tiene nombre: `append_Nil` y `append_Cons`. Los vamos a usar.

La inversa de una lista

```
fun inversa :: "'a list ⇒ 'a list" where
  "inversa [] = []"
| "inversa (x#xs) = inversa xs @ [x]"
```

La inversa de una lista

```
fun inversa :: "'a list ⇒ 'a list" where
  "inversa [] = []"
| "inversa (x#xs) = inversa xs @ [x]"

value "inversa [1::nat,2,3]"                    ⇝ [3, 2, 1]
```

La inversa de una lista

```
fun inversa :: "'a list ⇒ 'a list" where
  "inversa [] = []"
| "inversa (x#xs) = inversa xs @ [x]"
```

```
value "inversa [1::nat,2,3]"                    ⇝ [3, 2, 1]
```

- Se invierte la cola y se pega el primero al final.

La inversa de una lista

```
fun inversa :: "'a list ⇒ 'a list" where
  "inversa [] = []"
| "inversa (x#xs) = inversa xs @ [x]"
```

```
value "inversa [1::nat,2,3]"                    ⇝ [3, 2, 1]
```

- Se invierte la cola y se pega el primero al final.
- Es la definición de `rev` en la librería, con otro nombre. La reescribimos para poder demostrarla desde cero.

La inversa de una lista

```
fun inversa :: "'a list ⇒ 'a list" where
  "inversa [] = []"
| "inversa (x#xs) = inversa xs @ [x]"
```

```
value "inversa [1::nat,2,3]"                                    ⇝ [3, 2, 1]
```

- Se invierte la cola y se pega el primero al final.
- Es la definición de `rev` en la librería, con otro nombre. La reescribimos para poder demostrarla desde cero.

Cuesta un `@` por elemento: en total, del orden de n^2 pasos. Hay una versión lineal, con acumulador, que veremos como ejercicio.

inversa [x] = [x]: paso a paso

```
lemma "inversa [x] = [x]"
proof -
  have "inversa [x] = inversa (x#[])" by simp
  also have "... = (inversa []) @ [x]" by (simp only: inversa.simps(2))
  also have "... = [] @ [x]"           by (simp only: inversa.simps(1))
  also have "... = [x]"                by (simp only: append_Nil)
  finally show "inversa [x] = [x]" by simp
qed
```

inversa [x] = [x]: paso a paso

```
lemma "inversa [x] = [x]"
proof -
  have "inversa [x] = inversa (x#[])" by simp
  also have "... = (inversa []) @ [x]" by (simp only: inversa.simps(2))
  also have "... = [] @ [x]"           by (simp only: inversa.simps(1))
  also have "... = [x]"                by (simp only: append_Nil)
  finally show "inversa [x] = [x]" by simp
qed
```

- Cada paso dice qué ecuación usa, y solo esa: `simp only:`.

inversa [x] = [x]: paso a paso

```
lemma "inversa [x] = [x]"
proof -
  have "inversa [x] = inversa (x#[])" by simp
  also have "... = (inversa []) @ [x]" by (simp only: inversa.simps(2))
  also have "... = [] @ [x]"           by (simp only: inversa.simps(1))
  also have "... = [x]"                by (simp only: append_Nil)
  finally show "inversa [x] = [x]" by simp
qed
```

- Cada paso dice qué ecuación usa, y solo esa: `simp only:`.
- `...` es el lado derecho del paso anterior.

inversa [x] = [x]: paso a paso

```
lemma "inversa [x] = [x]"
proof -
  have "inversa [x] = inversa (x#[])" by simp
  also have "... = (inversa []) @ [x]" by (simp only: inversa.simps(2))
  also have "... = [] @ [x]"           by (simp only: inversa.simps(1))
  also have "... = [x]"                by (simp only: append_Nil)
  finally show "inversa [x] = [x]" by simp
qed
```

- Cada paso dice qué ecuación usa, y solo esa: `simp only:`.
- `...` es el lado derecho del paso anterior.
- `finally` encadena todos los pasos en una sola igualdad.

La misma, dos veces más corta

```
lemma "inversa [x] = [x]"
proof -
  have "inversa [x] = inversa (x#[])" by simp
  also have "... = (inversa []) @ [x]" by simp
  also have "... = [] @ [x]"           by simp
  also have "... = [x]"                by simp
  finally show "inversa [x] = [x]" by simp
qed
```

La misma, dos veces más corta

```
lemma "inversa [x] = [x]"
proof -
  have "inversa [x] = inversa (x#[])" by simp
  also have "... = (inversa []) @ [x]" by simp
  also have "... = [] @ [x]"           by simp
  also have "... = [x]"                by simp
  finally show "inversa [x] = [x]" by simp
qed

lemma "inversa [x] = [x]"
  by simp
```

La misma, dos veces más corta

```
lemma "inversa [x] = [x]"
proof -
  have "inversa [x] = inversa (x#[])" by simp
  also have "... = (inversa []) @ [x]" by simp
  also have "... = [] @ [x]"           by simp
  also have "... = [x]"                by simp
  finally show "inversa [x] = [x]" by simp
qed

lemma "inversa [x] = [x]"
  by simp
```

Las tres demuestran lo mismo.

¿Cuál de las tres escribir?

- La detallada, con `simp only:`, es la que enseña: se ve qué ecuación entra en cada paso. Úsela para entender, y cuando una demostración no le salga.

¿Cuál de las tres escribir?

- La detallada, con `simp only:`, es la que enseña: se ve qué ecuación entra en cada paso. Úsela para entender, y cuando una demostración no le salga.
- La intermedia conserva la estructura del argumento pero deja los detalles a `simp`.

¿Cuál de las tres escribir?

- La detallada, con `simp only:`, es la que enseña: se ve qué ecuación entra en cada paso. Úsela para entender, y cuando una demostración no le salga.
- La intermedia conserva la estructura del argumento pero deja los detalles a `simp`.
- La automática es la que queda en la teoría cuando ya no hay nada que discutir.

¿Cuál de las tres escribir?

- La detallada, con `simp only:`, es la que enseña: se ve qué ecuación entra en cada paso. Úsela para entender, y cuando una demostración no le salga.
- La intermedia conserva la estructura del argumento pero deja los detalles a `simp`.
- La automática es la que queda en la teoría cuando ya no hay nada que discutir.

Consejo

Escriba la detallada mientras piensa; deje la corta cuando ya entendió. Lo que no se puede es escribir la corta sin haber entendido.

El esquema `list.induct`

```
thm list.induct
```

$$\llbracket P\ []; \quad \bigwedge x1\ x2.\ P\ x2 \Longrightarrow P\ (x1\ \#\ x2) \rrbracket \Longrightarrow P\ xs$$

El esquema `list.induct`

```
thm list.induct
```

$$\llbracket P\ []; \quad \bigwedge x1\ x2.\ P\ x2 \Longrightarrow P\ (x1\ \#\ x2) \rrbracket \Longrightarrow P\ xs$$

- Para demostrar `P xs` para toda lista, basta:

El esquema `list.induct`

```
thm list.induct
```

$$\llbracket P\ [];\quad \bigwedge x1\ x2.\ P\ x2 \Longrightarrow P\ (x1\ \#\ x2) \rrbracket \Longrightarrow P\ xs$$

- Para demostrar `P xs` para toda lista, basta:
- caso base: `P []`;

El esquema `list.induct`

```
thm list.induct
```

$$\llbracket P\ [];\quad \bigwedge x1\ x2.\ P\ x2 \Longrightarrow P\ (x1\ \#\ x2) \rrbracket \Longrightarrow P\ xs$$

- Para demostrar `P xs` para toda lista, basta:
- caso base: `P []`;
- paso inductivo: si `P` vale para la cola `x2`, vale para `x1 # x2`.

El esquema `list.induct`

```
thm list.induct
```

$$\llbracket P\ [];\quad \bigwedge x1\ x2.\ P\ x2 \Longrightarrow P\ (x1\ \#\ x2) \rrbracket \Longrightarrow P\ xs$$

- Para demostrar `P xs` para toda lista, basta:
- caso base: `P []`;
- paso inductivo: si `P` vale para la cola `x2`, vale para `x1 # x2`.

Un caso por constructor: la misma forma que tiene la definición de las funciones.

Nuestra propia concatenación

```
fun conc :: "'a list ⇒ 'a list ⇒ 'a list" where
  "conc [] ys     = ys"
| "conc (x#xs) ys = x # (conc xs ys)"
```

Nuestra propia concatenación

```
fun conc :: "'a list ⇒ 'a list ⇒ 'a list" where
  "conc [] ys     = ys"
| "conc (x#xs) ys = x # (conc xs ys)"
```

```
value "conc [1::nat,2] [3,4]"                    ⇝ [1, 2, 3, 4]
```

Nuestra propia concatenación

```
fun conc :: "'a list ⇒ 'a list ⇒ 'a list" where
  "conc [] ys     = ys"
| "conc (x#xs) ys = x # (conc xs ys)"
```

```
value "conc [1::nat,2] [3,4]"                    ⇝ [1, 2, 3, 4]
```

- Es @ otra vez, escrita por nosotros.

Nuestra propia concatenación

```
fun conc :: "'a list ⇒ 'a list ⇒ 'a list" where
  "conc [] ys     = ys"
| "conc (x#xs) ys = x # (conc xs ys)"
```

```
value "conc [1::nat,2] [3,4]"                    ⇝ [1, 2, 3, 4]
```

- Es `@` otra vez, escrita por nosotros.
- Ventaja: las únicas dos cosas que Isabelle sabe de `conc` son esas dos ecuaciones. Todo lo demás hay que demostrarlo.

La concatenación es asociativa

```
lemma "conc xs (conc ys zs) = conc (conc xs ys) zs"
proof (induct xs)
  show "conc [] (conc ys zs) = conc (conc [] ys) zs" by simp
next
  fix x xs
  assume HI: "conc xs (conc ys zs) = conc (conc xs ys) zs"
  have "conc (x # xs) (conc ys zs) = x # (conc xs (conc ys zs))" by simp
  also have "... = x # (conc (conc xs ys) zs)" using HI by simp
  also have "... = conc (x # (conc xs ys)) zs" by simp
  also have "... = conc (conc (x # xs) ys) zs" by simp
  finally show "conc (x # xs) (conc ys zs) = conc (conc (x # xs) ys) zs" by simp
qed
```

La concatenación es asociativa

```
lemma "conc xs (conc ys zs) = conc (conc xs ys) zs"
proof (induct xs)
  show "conc [] (conc ys zs) = conc (conc [] ys) zs" by simp
next
  fix x xs
  assume HI: "conc xs (conc ys zs) = conc (conc xs ys) zs"
  have "conc (x # xs) (conc ys zs) = x # (conc xs (conc ys zs))" by simp
  also have "... = x # (conc (conc xs ys) zs)" using HI by simp
  also have "... = conc (x # (conc xs ys)) zs" by simp
  also have "... = conc (conc (x # xs) ys) zs" by simp
  finally show "conc (x # xs) (conc ys zs) = conc (conc (x # xs) ys) zs" by simp
qed
```

- `proof (induct xs)` abre exactamente los dos casos de `list.induct`.

La concatenación es asociativa

```
lemma "conc xs (conc ys zs) = conc (conc xs ys) zs"
proof (induct xs)
  show "conc [] (conc ys zs) = conc (conc [] ys) zs" by simp
next
  fix x xs
  assume HI: "conc xs (conc ys zs) = conc (conc xs ys) zs"
  have "conc (x # xs) (conc ys zs) = x # (conc xs (conc ys zs))" by simp
  also have "... = x # (conc (conc xs ys) zs)" using HI by simp
  also have "... = conc (x # (conc xs ys)) zs" by simp
  also have "... = conc (conc (x # xs) ys) zs" by simp
  finally show "conc (x # xs) (conc ys zs) = conc (conc (x # xs) ys) zs" by simp
qed
```

- `proof (induct xs)` abre exactamente los dos casos de `list.induct`.
- `fix x xs` nombra los pedazos del caso `x # xs`; `assume HI` le pone nombre a la hipótesis de inducción.

La concatenación es asociativa

```
lemma "conc xs (conc ys zs) = conc (conc xs ys) zs"
proof (induct xs)
  show "conc [] (conc ys zs) = conc (conc [] ys) zs" by simp
next
  fix x xs
  assume HI: "conc xs (conc ys zs) = conc (conc xs ys) zs"
  have "conc (x # xs) (conc ys zs) = x # (conc xs (conc ys zs))" by simp
  also have "... = x # (conc (conc xs ys) zs)" using HI by simp
  also have "... = conc (x # (conc xs ys)) zs" by simp
  also have "... = conc (conc (x # xs) ys) zs" by simp
  finally show "conc (x # xs) (conc ys zs) = conc (conc (x # xs) ys) zs" by simp
qed
```

- `proof (induct xs)` abre exactamente los dos casos de `list.induct`.
- `fix x xs` nombra los pedazos del caso `x # xs`; `assume HI` le pone nombre a la hipótesis de inducción.
- El único paso que usa `HI` es el segundo. Los demás son las ecuaciones de `conc`.

La misma, con los nombres que Isabelle ya da

```
lemma "conc xs (conc ys zs) = conc (conc xs ys) zs"
proof (induct xs)
  case Nil
  then show ?case by auto
next
  case (Cons x xs)
  then show ?case
  proof -
    have "conc (x # xs) (conc ys zs) = x # (conc xs (conc ys zs))" by simp
    also have "... = x # (conc (conc xs ys) zs)" by (simp add: Cons)
    also have "... = conc (x # (conc xs ys)) zs" by simp
    also have "... = conc (conc (x # xs) ys) zs" by simp
    finally show ?thesis by simp
  qed
qed
```

La misma, con los nombres que Isabelle ya da

```
lemma "conc xs (conc ys zs) = conc (conc xs ys) zs"
proof (induct xs)
  case Nil
  then show ?case by auto
next
  case (Cons x xs)
  then show ?case
  proof -
    have "conc (x # xs) (conc ys zs) = x # (conc xs (conc ys zs))" by simp
    also have "... = x # (conc (conc xs ys) zs)" by (simp add: Cons)
    also have "... = conc (x # (conc xs ys)) zs" by simp
    also have "... = conc (conc (x # xs) ys) zs" by simp
    finally show ?thesis by simp
  qed
qed
```

- `case (Cons x xs)` hace el `fix` y el `assume` de un tirón: los nombres salen del `datatype`.

La misma, con los nombres que Isabelle ya da

```
lemma "conc xs (conc ys zs) = conc (conc xs ys) zs"
proof (induct xs)
  case Nil
  then show ?case by auto
next
  case (Cons x xs)
  then show ?case
  proof -
    have "conc (x # xs) (conc ys zs) = x # (conc xs (conc ys zs))" by simp
    also have "... = x # (conc (conc xs ys) zs)" by (simp add: Cons)
    also have "... = conc (x # (conc xs ys)) zs" by simp
    also have "... = conc (conc (x # xs) ys) zs" by simp
    finally show ?thesis by simp
  qed
qed
```

- `case (Cons x xs)` hace el `fix` y el `assume` de un tirón: los nombres salen del `datatype`.
- `?case` es lo que hay que demostrar en este caso; `?thesis`, lo que anuncia el `show`.

Y la automática

```
lemma "conc xs (conc ys zs) = conc (conc xs ys) zs"
  by (induct xs) auto
```

Y la automática

```
lemma "conc xs (conc ys zs) = conc (conc xs ys) zs"
  by (induct xs) auto
```

Una línea. Pero es la misma inducción: `induct xs` abre los dos casos y `auto` los cierra con las ecuaciones de `conc`.

Y la automática

```
lemma "conc xs (conc ys zs) = conc (conc xs ys) zs"
  by (induct xs) auto
```

Una línea. Pero es la misma inducción: `induct xs` abre los dos casos y `auto` los cierra con las ecuaciones de `conc`.

Si `auto` no cierra alguno, vuelva a la versión larga: ahí se ve en qué paso se atasca.

¿Y conmutativa?

```
lemma "conc xs ys = conc ys xs"
quickcheck
oops
```

¿Y conmutativa?

```
lemma "conc xs ys = conc ys xs"
quickcheck
oops
```

- `quickcheck` busca un contraejemplo probando valores, antes de que usted pierda la tarde.

¿Y conmutativa?

```
lemma "conc xs ys = conc ys xs"
quickcheck
oops
```

- `quickcheck` busca un contraejemplo probando valores, antes de que usted pierda la tarde.
- Lo encuentra enseguida: `conc [a] [b] = [a,b]` pero `conc [b] [a] = [b,a]`.

¿Y conmutativa?

```
lemma "conc xs ys = conc ys xs"
quickcheck
oops
```

- `quickcheck` busca un contraejemplo probando valores, antes de que usted pierda la tarde.
- Lo encuentra enseguida: `conc [a] [b] = [a,b]` pero `conc [b] [a] = [b,a]`.
- `oops` abandona el lema. Es falso: no hay nada que demostrar.

¿Y conmutativa?

```
lemma "conc xs ys = conc ys xs"
quickcheck
oops
```

- `quickcheck` busca un contraejemplo probando valores, antes de que usted pierda la tarde.
- Lo encuentra enseguida: `conc [a] [b] = [a,b]` pero `conc [b] [a] = [b,a]`.
- `oops` abandona el lema. Es falso: no hay nada que demostrar.

Hábito: antes de demostrar, pregunte si es verdad.

La lista vacía es elemento neutro

```
lemma "conc xs [] = xs"
proof (induct xs)
  show "conc [] [] = []" by simp
next
  fix x xs
  assume HI: "conc xs [] = xs"
  have "conc (x # xs) [] = x # (conc xs [])" by simp
  also have "... = x # xs" using HI by simp
  finally show "conc (x # xs) [] = x # xs" by simp
qed
```

La lista vacía es elemento neutro

```
lemma "conc xs [] = xs"
proof (induct xs)
  show "conc [] [] = []" by simp
next
  fix x xs
  assume HI: "conc xs [] = xs"
  have "conc (x # xs) [] = x # (conc xs [])" by simp
  also have "... = x # xs" using HI by simp
  finally show "conc (x # xs) [] = x # xs" by simp
qed
```

Lo importante de este ejemplo

`conc [] ys = ys` es una de las dos ecuaciones: sale gratis.

`conc xs [] = xs` no aparece en la definición: hay que demostrarla, y hace falta inducción.

La longitud

```
fun longitud :: "'a list ⇒ nat" where
  "longitud []     = 0"
| "longitud (x#xs) = 1 + longitud xs"
```

La longitud

```
fun longitud :: "'a list ⇒ nat" where
  "longitud []     = 0"
| "longitud (x#xs) = 1 + longitud xs"

value "longitud [1::nat,2,3]"                    ⇝ 3
```

La longitud

```
fun longitud :: "'a list ⇒ nat" where
  "longitud []     = 0"
| "longitud (x#xs) = 1 + longitud xs"

value "longitud [1::nat,2,3]"                                    ⇝ 3
```

Otra vez: una ecuación por constructor, y la recursión sobre la cola.

La longitud de una concatenación

```
lemma "longitud (conc xs ys) = longitud xs + longitud ys"
proof (induct xs)
  show "longitud (conc [] ys) = longitud [] + longitud ys" by simp
next
  fix x xs
  assume HI: "longitud (conc xs ys) = longitud xs + longitud ys"
  have "longitud (conc (x # xs) ys) = longitud (x # (conc xs ys))" by simp
  also have "... = 1 + longitud (conc xs ys)" by simp
  also have "... = 1 + longitud xs + longitud ys" using HI by simp
  also have "... = longitud (x # xs) + longitud ys" by simp
  finally show "longitud (conc (x # xs) ys) = longitud (x # xs) + longitud ys" by simp
qed
```

Dos ejercicios sobre la inversa

```
lemma inv_conc [simp]: "inversa (xs @ ys) = (inversa ys) @ (inversa xs)"
  oops

theorem inv_inv [simp]: "inversa (inversa xs) = xs"
  oops
```

Dos ejercicios sobre la inversa

```
lemma inv_conc [simp]: "inversa (xs @ ys) = (inversa ys) @ (inversa xs)"
  oops

theorem inv_inv [simp]: "inversa (inversa xs) = xs"
  oops
```

- Los dos salen por inducción sobre `xs`.

Dos ejercicios sobre la inversa

```
lemma inv_conc [simp]: "inversa (xs @ ys) = (inversa ys) @ (inversa xs)"
  oops

theorem inv_inv [simp]: "inversa (inversa xs) = xs"
  oops
```

- Los dos salen por inducción sobre `xs`.
- El segundo necesita el primero: cuando lo demuestren, el atributo `[simp]` hará que el segundo salga solo.

Dos ejercicios sobre la inversa

```
lemma inv_conc [simp]: "inversa (xs @ ys) = (inversa ys) @ (inversa xs)"
  oops

theorem inv_inv [simp]: "inversa (inversa xs) = xs"
  oops
```

- Los dos salen por inducción sobre `xs`.
- El segundo necesita el primero: cuando lo demuestren, el atributo `[simp]` hará que el segundo salga solo.
- `oops` los deja anunciados y sin demostrar. Ese es el trabajo de ustedes.

Dos ejercicios sobre la inversa

```
lemma inv_conc [simp]: "inversa (xs @ ys) = (inversa ys) @ (inversa xs)"
  oops

theorem inv_inv [simp]: "inversa (inversa xs) = xs"
  oops
```

- Los dos salen por inducción sobre `xs`.
- El segundo necesita el primero: cuando lo demuestren, el atributo `[simp]` hará que el segundo salga solo.
- `oops` los deja anunciados y sin demostrar. Ese es el trabajo de ustedes.

Sugerencia: empiece con `by (induct xs) auto` y, si no sale, escriba la cadena de igualdades a mano. Son A4 y A5 de la relación de ejercicios.

Naturales:

`EjemplosTipoNumerosNaturales.thy`

Los naturales también son un tipo recursivo

```
datatype nat = 0 | Suc nat
```

Los naturales también son un tipo recursivo

```
datatype nat = 0 | Suc nat
```

- Dos constructores, como las listas: `0` y `Suc n`.

Los naturales también son un tipo recursivo

```
datatype nat = 0 | Suc nat
```

- Dos constructores, como las listas: `0` y `Suc n`.
- 2 es notación para `Suc (Suc 0)`:

Los naturales también son un tipo recursivo

```
datatype nat = 0 | Suc nat
```

- Dos constructores, como las listas: `0` y `Suc n`.
- 2 es notación para `Suc (Suc 0)`:

```
lemma "(Suc (Suc 0)) == 2" by linarith
```

Los naturales también son un tipo recursivo

```
datatype nat = 0 | Suc nat
```

- Dos constructores, como las listas: `0` y `Suc n`.
- `2` es notación para `Suc (Suc 0)`:

```
lemma "(Suc (Suc 0)) == 2" by linarith
```

```
thm nat.induct
```

$$\llbracket P\ 0;\ \bigwedge n.\ P\ n \Longrightarrow P\ (\mathrm{Suc}\ n) \rrbracket \Longrightarrow P\ n$$

Los naturales también son un tipo recursivo

```
datatype nat = 0 | Suc nat
```

- Dos constructores, como las listas: `0` y `Suc n`.
- 2 es notación para `Suc (Suc 0)`:

```
lemma "(Suc (Suc 0)) == 2" by linarith
```

```
thm nat.induct
```

$$\llbracket P\ 0;\ \bigwedge n.\ P\ n \Longrightarrow P\ (Suc\ n) \rrbracket \Longrightarrow P\ n$$

Es el método de inducción de los naturales, y tiene la misma forma que `list.induct`.

La suma de los n primeros pares

```
fun Suma_pares :: "nat ⇒ nat" where
  "Suma_pares 0       = 0"
| "Suma_pares (Suc n) = (Suma_pares n) + 2*(n+1)"
```

La suma de los n primeros pares

```
fun Suma_pares :: "nat ⇒ nat" where
  "Suma_pares 0       = 0"
| "Suma_pares (Suc n) = (Suma_pares n) + 2*(n+1)"

value "Suma_pares 3"                                    ⇝ 12    2 + 4 + 6
```

La suma de los n primeros pares

```
fun Suma_pares :: "nat ⇒ nat" where
  "Suma_pares 0       = 0"
| "Suma_pares (Suc n) = (Suma_pares n) + 2*(n+1)"
```

```
value "Suma_pares 3"
```
$\rightsquigarrow 12 \qquad 2+4+6$

Lo que queremos demostrar

```
Suma_pares n = n*(n+1)
```

La suma de los n primeros pares

```
fun Suma_pares :: "nat ⇒ nat" where
  "Suma_pares 0       = 0"
| "Suma_pares (Suc n) = (Suma_pares n) + 2*(n+1)"
```

```
value "Suma_pares 3"
```
$\rightsquigarrow 12 \qquad 2+4+6$

Lo que queremos demostrar

```
Suma_pares n = n*(n+1)
```

Vale para 3: $12 = 3 \cdot 4$. Pero eso no demuestra nada: hay infinitos naturales.

La demostración, por inducción

```
lemma suma_pares: "Suma_pares n = n*(n+1)"
proof (induct n)
  case 0
  have "(Suma_pares 0) = 0 * (0 + 1)" by simp
  then show ?case by simp
next
  case (Suc n)
  assume HI: "Suma_pares n = n * (n + 1)"
  show "Suma_pares (Suc n) = Suc n * (Suc n + 1)"
  proof -
    have "Suma_pares (Suc n) = (Suma_pares n) + 2*(n+1)" by simp
    also have "... = (Suma_pares n) + 2*n + 2"  by simp
    also have "... = n*(n+1) + 2*n + 2" using HI by simp
    also have "... = n*(n+1) + 2*(n + 1)"       by simp
    also have "... = (n+2) * (n+1)"             by simp
    finally show ?thesis by simp
  qed
qed
```

¿Dónde entra la inducción?

- `case 0` y `case (Suc n)` son los dos casos de `nat.induct`.

¿Dónde entra la inducción?

- `case 0` y `case (Suc n)` son los dos casos de `nat.induct`.
- De los cinco pasos de la cadena, uno solo usa la hipótesis: el tercero, el del `using HI`.

¿Dónde entra la inducción?

- `case 0` y `case (Suc n)` son los dos casos de `nat.induct`.
- De los cinco pasos de la cadena, uno solo usa la hipótesis: el tercero, el del `using HI`.
- Los otros cuatro son la ecuación de `Suma_pares` y aritmética que `simp` hace sola.

¿Dónde entra la inducción?

- `case 0` y `case (Suc n)` son los dos casos de `nat.induct`.
- De los cinco pasos de la cadena, uno solo usa la hipótesis: el tercero, el del `using HI`.
- Los otros cuatro son la ecuación de `Suma_pares` y aritmética que `simp` hace sola.

Esa es la forma de casi toda demostración por inducción: un paso sustituye la hipótesis, el resto es cuenta.

Impares, naturales, y una identidad entre las tres

```
fun Suma_impares :: "nat ⇒ nat" where
  "Suma_impares 0       = 0"
| "Suma_impares (Suc n) = (Suma_impares n) + 2*n+1"

lemma suma_impares: "Suma_impares n = n*n"
proof (induct n)
  show "Suma_impares 0 = 0 * 0" by simp
next
  fix n
  assume "Suma_impares n = n * n"
  show "Suma_impares (Suc n) = Suc n * Suc n"
    by (simp add: <Suma_impares n = n * n>)
qed
```

Impares, naturales, y una identidad entre las tres

```
fun Suma_impares :: "nat ⇒ nat" where
  "Suma_impares 0       = 0"
| "Suma_impares (Suc n) = (Suma_impares n) + 2*n+1"

lemma suma_impares: "Suma_impares n = n*n"
proof (induct n)
  show "Suma_impares 0 = 0 * 0" by simp
next
  fix n
  assume "Suma_impares n = n * n"
  show "Suma_impares (Suc n) = Suc n * Suc n"
    by (simp add: <Suma_impares n = n * n>)
qed
```

- Cuando la hipótesis no tiene nombre, se la cita por su enunciado, entre ‹ › llamados cartuchos.

Impares, naturales, y una identidad entre las tres

```
fun Suma_impares :: "nat ⇒ nat" where
  "Suma_impares 0       = 0"
| "Suma_impares (Suc n) = (Suma_impares n) + 2*n+1"

lemma suma_impares: "Suma_impares n = n*n"
proof (induct n)
  show "Suma_impares 0 = 0 * 0" by simp
next
  fix n
  assume "Suma_impares n = n * n"
  show "Suma_impares (Suc n) = Suc n * Suc n"
    by (simp add: <Suma_impares n = n * n>)
qed
```

- Cuando la hipótesis no tiene nombre, se la cita por su enunciado, entre ‹ › llamados cartuchos.
- En jEdit los cartuchos salen escribiendo \<open> y \<close>, o copiando el enunciado del objetivo. Más fácil: póngale nombre a la hipótesis con `assume HI:`.

Los pares y los impares, juntos

```
fun Suma_naturales :: "nat ⇒ nat" where
  "Suma_naturales 0       = 0"
| "Suma_naturales (Suc n) = (Suma_naturales n) + n+1"

lemma suma_naturales: "2*(Suma_naturales n) = n*(n+1)"
  by (induct n) auto

lemma "Suma_naturales (2*n) = Suma_pares n + (Suma_impares n)"
proof -
  have "2*Suma_naturales (2*n) = 2*(Suma_pares n + (Suma_impares n))"
    using suma_pares suma_impares suma_naturales by simp
  thus ?thesis by simp
qed
```

Los pares y los impares, juntos

```
fun Suma_naturales :: "nat ⇒ nat" where
  "Suma_naturales 0       = 0"
| "Suma_naturales (Suc n) = (Suma_naturales n) + n+1"

lemma suma_naturales: "2*(Suma_naturales n) = n*(n+1)"
  by (induct n) auto

lemma "Suma_naturales (2*n) = Suma_pares n + (Suma_impares n)"
proof -
  have "2*Suma_naturales (2*n) = 2*(Suma_pares n + (Suma_impares n))"
    using suma_pares suma_impares suma_naturales by simp
  thus ?thesis by simp
qed
```

- Aquí no hay inducción: se usan los tres lemas ya demostrados.

Los pares y los impares, juntos

```
fun Suma_naturales :: "nat ⇒ nat" where
  "Suma_naturales 0       = 0"
| "Suma_naturales (Suc n) = (Suma_naturales n) + n+1"

lemma suma_naturales: "2*(Suma_naturales n) = n*(n+1)"
  by (induct n) auto

lemma "Suma_naturales (2*n) = Suma_pares n + (Suma_impares n)"
proof -
  have "2*Suma_naturales (2*n) = 2*(Suma_pares n + (Suma_impares n))"
    using suma_pares suma_impares suma_naturales by simp
  thus ?thesis by simp
qed
```

- Aquí no hay inducción: se usan los tres lemas ya demostrados.
- Por eso les pusimos nombre. Un lema con nombre es un lema reutilizable.

Par, por su significado

n es par si existe m con $n = m + m$.

```
definition par :: "nat ⇒ bool" where
  "par n ≡ ∃m. n = m+m"
```

Par, por su significado

n es par si existe m con $n = m + m$.

```
definition par :: "nat ⇒ bool" where
  "par n ≡ ∃m. n = m+m"
```

- `definition`, no `fun`: no hay recursión, hay una descripción.

Par, por su significado

n es par si existe m con $n = m + m$.

```
definition par :: "nat ⇒ bool" where
  "par n ≡ ∃m. n = m+m"
```

- `definition`, no `fun`: no hay recursión, hay una descripción.
- El nombre de la ecuación es `par_def`, y hay que pedirlo: `simp add: par_def`.

Par, por su significado

n es par si existe m con $n = m + m$.

```
definition par :: "nat ⇒ bool" where
  "par n ≡ ∃m. n = m+m"
```

- `definition`, no `fun`: no hay recursión, hay una descripción.
- El nombre de la ecuación es `par_def`, y hay que pedirlo: `simp add: par_def`.

Ojo: esta definición no se puede evaluar

```
value "par 10"
```

Wellsortedness error ... Type nat not of sort enum

Isabelle tendría que recorrer todos los naturales buscando ese `m`.

n*(n+1) es par

```
lemma
shows "par (n*(n+1))"
proof (induct n)
  show "par (0*(0+1))" by (simp add: par_def)
next
  fix n
  assume "par (n*(n+1))"
  hence "∃m. n*(n+1) = m+m" by (simp add: par_def)
  then obtain m where m: "n*(n+1) = m+m" by (rule exE)
  hence "(Suc n)*((Suc n)+1) = (m+n+1)+(m+n+1)" by auto
  hence "∃m. (Suc n)*((Suc n)+1) = m+m" by (rule exI)
  thus "par ((Suc n)*((Suc n)+1))" by (simp add: par_def)
qed
```

n*(n+1) es par

```
lemma
shows "par (n*(n+1))"
proof (induct n)
  show "par (0*(0+1))" by (simp add: par_def)
next
  fix n
  assume "par (n*(n+1))"
  hence "∃m. n*(n+1) = m+m" by (simp add: par_def)
  then obtain m where m: "n*(n+1) = m+m" by (rule exE)
  hence "(Suc n)*((Suc n)+1) = (m+n+1)+(m+n+1)" by auto
  hence "∃m. (Suc n)*((Suc n)+1) = m+m" by (rule exI)
  thus "par ((Suc n)*((Suc n)+1))" by (simp add: par_def)
qed
```

- `obtain m where ...` le pone nombre al testigo que el existencial promete: es `exE`.

n*(n+1) es par

```
lemma
shows "par (n*(n+1))"
proof (induct n)
  show "par (0*(0+1))" by (simp add: par_def)
next
  fix n
  assume "par (n*(n+1))"
  hence "∃m. n*(n+1) = m+m" by (simp add: par_def)
  then obtain m where m: "n*(n+1) = m+m" by (rule exE)
  hence "(Suc n)*((Suc n)+1) = (m+n+1)+(m+n+1)" by auto
  hence "∃m. (Suc n)*((Suc n)+1) = m+m" by (rule exI)
  thus "par ((Suc n)*((Suc n)+1))" by (simp add: par_def)
qed
```

- `obtain m where ...` le pone nombre al testigo que el existencial promete: es `exE`.
- Al final se construye el testigo nuevo, `m+n+1`, y se vuelve a cerrar el existencial con `exI`.

n*(n+1) es par

```
lemma
shows "par (n*(n+1))"
proof (induct n)
  show "par (0*(0+1))" by (simp add: par_def)
next
  fix n
  assume "par (n*(n+1))"
  hence "∃m. n*(n+1) = m+m" by (simp add: par_def)
  then obtain m where m: "n*(n+1) = m+m" by (rule exE)
  hence "(Suc n)*((Suc n)+1) = (m+n+1)+(m+n+1)" by auto
  hence "∃m. (Suc n)*((Suc n)+1) = m+m" by (rule exI)
  thus "par ((Suc n)*((Suc n)+1))" by (simp add: par_def)
qed
```

- `obtain m where ...` le pone nombre al testigo que el existencial promete: es `exE`.
- Al final se construye el testigo nuevo, `m+n+1`, y se vuelve a cerrar el existencial con `exI`.

exE y exI son las reglas de la Parte III, con cuantificadores. Aquí van de adelanto.

La suma de dos pares es par

```
lemma shows "par n ∧ par m ⟶ (par (n+m))"
proof (induct n)
  show "(par 0) ∧ (par m) ⟶ par (0 + m)" by auto
next
  fix n
  assume "par n ∧ par m ⟶ par (n + m)"
  show   "par (Suc n) ∧ par m ⟶ par (Suc n + m)"
  proof (rule impI)
    assume "par (Suc n) ∧ par m"
    hence  "(∃m1. Suc n = m1+m1) ∧ (∃m2. m = m2+m2)" by (simp add: par_def)
    then obtain m1 m2 where "Suc n = m1+m1 ∧ m = m2+m2" by auto
    hence "Suc (n + m) = (m1+m2)+(m1+m2)"  by auto
    hence "∃k. Suc (n + m) = k+k" by auto
    thus  "par (Suc n + m)"  by (simp add: par_def)
  qed
qed
```

La suma de dos pares es par

```
lemma shows "par n ∧ par m ⟶ (par (n+m))"
proof (induct n)
  show "(par 0) ∧ (par m) ⟶ par (0 + m)" by auto
next
  fix n
  assume "par n ∧ par m ⟶ par (n + m)"
  show   "par (Suc n) ∧ par m ⟶ par (Suc n + m)"
  proof (rule impI)
    assume "par (Suc n) ∧ par m"
    hence  "(∃m1. Suc n = m1+m1) ∧ (∃m2. m = m2+m2)" by (simp add: par_def)
    then obtain m1 m2 where "Suc n = m1+m1 ∧ m = m2+m2" by auto
    hence "Suc (n + m) = (m1+m2)+(m1+m2)"  by auto
    hence "∃k. Suc (n + m) = k+k" by auto
    thus  "par (Suc n + m)"  by (simp add: par_def)
  qed
qed
```

`rule impI` es la misma regla de la Parte I: para demostrar `A ⟶ B`, suponga `A` y demuestre `B`.

Isabelle ya tenía una: even

```
lemma
fixes n :: "nat"
shows "even (n*(n+1))"
  by auto
```

Isabelle ya tenía una: `even`

```
lemma
fixes n :: "nat"
shows "even (n*(n+1))"
  by auto
```

Una línea, porque `even` sí es ejecutable y `simp` sabe de ella.

Y las dos definiciones coinciden

```
lemma
shows "par n = even n"
proof -
  have "par n = (∃m. n = m+m)" by (simp add: par_def)
  thus "par n = even n" by presburger
qed
```

Y las dos definiciones coinciden

```
lemma
shows "par n = even n"
proof -
  have "par n = (∃m. n = m+m)" by (simp add: par_def)
  thus "par n = even n" by presburger
qed
```

`presburger` decide la aritmética lineal de los enteros. Es un procedimiento de decisión: o lo resuelve, o dice que no.

Otra forma: por patrones

```
fun par1 :: "nat ⇒ bool" where
  "par1 0             = True"
| "par1 (Suc 0)       = False"
| "par1 (Suc (Suc n)) = par1 n"
```

Otra forma: por patrones

```
fun par1 :: "nat ⇒ bool" where
  "par1 0             = True"
| "par1 (Suc 0)       = False"
| "par1 (Suc (Suc n)) = par1 n"

value "par1 31"                                    ⇝ False
```

Otra forma: por patrones

```
fun par1 :: "nat ⇒ bool" where
  "par1 0             = True"
| "par1 (Suc 0)       = False"
| "par1 (Suc (Suc n)) = par1 n"
```

```
value "par1 31"                                    ⇝ False
```

- Calcula perfectamente: baja de dos en dos hasta 0 o 1.

Otra forma: por patrones

```
fun par1 :: "nat ⇒ bool" where
  "par1 0             = True"
| "par1 (Suc 0)       = False"
| "par1 (Suc (Suc n)) = par1 n"
```

```
value "par1 31"
```
⇝ False

- Calcula perfectamente: baja de dos en dos hasta 0 o 1.
- Pero, como dice la teoría, no es apropiada para demostrar: la recursión salta dos pasos, y la hipótesis de inducción que da `nat.induct` solo retrocede uno.

Calcular y describir no es lo mismo

`fun par1`

- Dice cómo se calcula.
- `value` la ejecuta.
- Demostrar con ella es incómodo.

Calcular y describir no es lo mismo

`fun par1`

- Dice cómo se calcula.
- `value` la ejecuta.
- Demostrar con ella es incómodo.

`definition par`

- Dice qué significa.
- `value` no puede con ella.
- Demostrar con ella es natural.

Calcular y describir no es lo mismo

`fun par1`

- Dice cómo se calcula.
- `value` la ejecuta.
- Demostrar con ella es incómodo.

`definition par`

- Dice qué significa.
- `value` no puede con ella.
- Demostrar con ella es natural.

Las dos son legítimas. Se elige según lo que uno vaya a hacer con la función.

Todo tipo inductivo trae su inducción

Así cierra la teoría: todo tipo inductivo define un método de prueba, la inducción estructural.

Todo tipo inductivo trae su inducción

Así cierra la teoría: todo tipo inductivo define un método de prueba, la inducción estructural.

thm list.induct

```
⟦P []; ⋀x1 x2. P x2 ⟹ P (x1 # x2)⟧ ⟹ P xs
```

thm nat.induct

```
⟦P 0; ⋀n. P n ⟹ P (Suc n)⟧ ⟹ P n
```

Todo tipo inductivo trae su inducción

Así cierra la teoría: todo tipo inductivo define un método de prueba, la inducción estructural.

thm list.induct

```
⟦P [];
 ⋀x1 x2. P x2 ⟹ P (x1 # x2)⟧
⟹ P xs
```

thm nat.induct

```
⟦P 0;
 ⋀n. P n ⟹ P (Suc n)⟧
⟹ P n
```

- Isabelle genera solo el principio al aceptar el `datatype`: un caso por constructor.

Todo tipo inductivo trae su inducción

Así cierra la teoría: todo tipo inductivo define un método de prueba, la inducción estructural.

thm list.induct

```
⟦P [];
 ⋀x1 x2. P x2 ⟹ P (x1 # x2)⟧
⟹ P xs
```

thm nat.induct

```
⟦P 0;
 ⋀n. P n ⟹ P (Suc n)⟧
⟹ P n
```

- Isabelle genera solo el principio al aceptar el `datatype`: un caso por constructor.
- Cada argumento recursivo del constructor trae su hipótesis de inducción: la cola en las listas, el predecesor en los naturales.

Todo tipo inductivo trae su inducción

Así cierra la teoría: todo tipo inductivo define un método de prueba, la inducción estructural.

thm list.induct

```
⟦P [];
 ⋀x1 x2. P x2 ⟹ P (x1 # x2)⟧
⟹ P xs
```

thm nat.induct

```
⟦P 0;
 ⋀n. P n ⟹ P (Suc n)⟧
⟹ P n
```

- Isabelle genera solo el principio al aceptar el `datatype`: un caso por constructor.
- Cada argumento recursivo del constructor trae su hipótesis de inducción: la cola en las listas, el predecesor en los naturales.
- Lo mismo vale para cualquier tipo que usted defina: en el Tema 6, los árboles.

Más listas:

`OtrosEjemplosListas.thy`

Repetir un elemento `n` veces

```
primrec repite :: "nat ⇒ 'a ⇒ 'a list" where
  "repite 0 x       = []"
| "repite (Suc n) x = x # (repite n x)"
```

Repetir un elemento `n` veces

```
primrec repite :: "nat ⇒ 'a ⇒ 'a list" where
  "repite 0 x       = []"
| "repite (Suc n) x = x # (repite n x)"
```

```
value "repite 3 (5::nat)"                                    ⇝ [5, 5, 5]
```

Repetir un elemento `n` veces

```
primrec repite :: "nat ⇒ 'a ⇒ 'a list" where
  "repite 0 x       = []"
| "repite (Suc n) x = x # (repite n x)"
```

```
value "repite 3 (5::nat)"
```
⇝ `[5, 5, 5]`

- Dos argumentos, pero la recursión va solo sobre `n`.

Repetir un elemento `n` veces

```
primrec repite :: "nat ⇒ 'a ⇒ 'a list" where
  "repite 0 x       = []"
| "repite (Suc n) x = x # (repite n x)"
```

```
value "repite 3 (5::nat)"                                    ⇝ [5, 5, 5]
```

- Dos argumentos, pero la recursión va solo sobre `n`.
- `primrec`: recursión primitiva, una ecuación por constructor. `fun` admite más, como las tres de coge que vienen ahora.

Repetir un elemento `n` veces

```
primrec repite :: "nat ⇒ 'a ⇒ 'a list" where
  "repite 0 x       = []"
| "repite (Suc n) x = x # (repite n x)"
```

```
value "repite 3 (5::nat)"                                  ⇝ [5, 5, 5]
```

- Dos argumentos, pero la recursión va solo sobre `n`.
- `primrec`: recursión primitiva, una ecuación por constructor. `fun` admite más, como las tres de coge que vienen ahora.

La teoría vuelve a definir `longitud` y `conc`: así se puede abrir sola, sin la primera.

La lista repetida tiene longitud `n`

```
lemma "longitud (repite n x) = n"
proof (induct n)
  show "longitud (repite 0 x) = 0" by simp
next
  fix n
  assume HI: "longitud (repite n x) = n"
  have "longitud (repite (Suc n) x) = longitud (x # (repite n x))" by simp
  also have "... = 1 + longitud (repite n x)" by simp
  also have "... = 1 + n" using HI by simp
  finally show "longitud (repite (Suc n) x) = Suc n" by simp
qed
```

La lista repetida tiene longitud n

```
lemma "longitud (repite n x) = n"
proof (induct n)
  show "longitud (repite 0 x) = 0" by simp
next
  fix n
  assume HI: "longitud (repite n x) = n"
  have "longitud (repite (Suc n) x) = longitud (x # (repite n x))" by simp
  also have "... = 1 + longitud (repite n x)" by simp
  also have "... = 1 + n" using HI by simp
  finally show "longitud (repite (Suc n) x) = Suc n" by simp
qed
```

La inducción va por n, el argumento que recursa. La x viaja intacta.

Tomar y quitar los primeros n

```
fun coge :: "nat ⇒ 'a list ⇒ 'a list" where
  "coge n []           = []"
| "coge 0 xs           = []"
| "coge (Suc n) (x#xs) = x # (coge n xs)"

fun elimina :: "nat ⇒ 'a list ⇒ 'a list" where
  "elimina n []           = []"
| "elimina 0 xs           = xs"
| "elimina (Suc n) (x#xs) = elimina n xs"
```

Tres ecuaciones, no dos

```
value "coge 2 [1::nat,2,3,4]"
```
⇝ [1, 2]

```
value "elimina 2 [2::nat,3,2,2]"
```
⇝ [2, 2]

Tres ecuaciones, no dos

```
value "coge 2 [1::nat,2,3,4]"
```
⇝ [1, 2]

```
value "elimina 2 [2::nat,3,2,2]"
```
⇝ [2, 2]

- Tres ecuaciones, no dos: se reparten sobre los dos argumentos a la vez.

Tres ecuaciones, no dos

```
value "coge 2 [1::nat,2,3,4]"          ⇝ [1, 2]
value "elimina 2 [2::nat,3,2,2]"       ⇝ [2, 2]
```

- Tres ecuaciones, no dos: se reparten sobre los dos argumentos a la vez.
- Las dos primeras se solapan en `coge 0 []`. Isabelle toma la primera que encaje, como Haskell.

Cada función trae su propia inducción

```
thm coge.induct
```

```
⟦⋀n. P n [];
 ⋀x xs. P 0 (x#xs);
 ⋀n x xs. P n xs ⟹ P (Suc n) (x#xs)⟧ ⟹ P n xs
```

Cada función trae su propia inducción

```
thm coge.induct
```

```
⟦⋀n. P n [];
 ⋀x xs. P 0 (x#xs);
 ⋀n x xs. P n xs ⟹ P (Suc n) (x#xs)⟧ ⟹ P n xs
```

- Un caso por ecuación de la definición, y la hipótesis de inducción viene de la llamada recursiva.

Cada función trae su propia inducción

```
thm coge.induct
```

```
⟦⋀n. P n [];
 ⋀x xs. P 0 (x#xs);
 ⋀n x xs. P n xs ⟹ P (Suc n) (x#xs)⟧ ⟹ P n xs
```

- Un caso por ecuación de la definición, y la hipótesis de inducción viene de la llamada recursiva.
- Isabelle lo genera al aceptar el `fun`: es la prueba de que la recursión termina.

Cada función trae su propia inducción

```
thm coge.induct
```

```
⟦⋀n. P n [];
 ⋀x xs. P 0 (x#xs);
 ⋀n x xs. P n xs ⟹ P (Suc n) (x#xs)⟧ ⟹ P n xs
```

- Un caso por ecuación de la definición, y la hipótesis de inducción viene de la llamada recursiva.
- Isabelle lo genera al aceptar el `fun`: es la prueba de que la recursión termina.
- `induct xs` no serviría aquí: hay que bajar los dos argumentos a la vez.

Definir por casos, en vez de por patrones

```
fun elimina1 :: "nat ⇒ 'a list ⇒ 'a list" where
  elimina1_Nil:  "elimina1 n [] = []"
| elimina1_Cons: "elimina1 n (x#xs) = (case n of
                                         0 => x#xs
                                       | Suc(m) => elimina1 m xs)"
```

Definir por casos, en vez de por patrones

```
fun elimina1 :: "nat ⇒ 'a list ⇒ 'a list" where
  elimina1_Nil:  "elimina1 n [] = []"
| elimina1_Cons: "elimina1 n (x#xs) = (case n of
                                         0 => x#xs
                                       | Suc(m) => elimina1 m xs)"
```

- Dos ecuaciones en vez de tres: el segundo argumento se distingue por patrón, y el primero dentro de la ecuación, con case.

Definir por casos, en vez de por patrones

```
fun elimina1 :: "nat ⇒ 'a list ⇒ 'a list" where
  elimina1_Nil:  "elimina1 n [] = []"
| elimina1_Cons: "elimina1 n (x#xs) = (case n of
                                         0 => x#xs
                                       | Suc(m) => elimina1 m xs)"
```

- Dos ecuaciones en vez de tres: el segundo argumento se distingue por patrón, y el primero dentro de la ecuación, con `case`.
- La llamada recursiva está dentro del `case`: solo ocurre cuando `n = Suc m`.

Definir por casos, en vez de por patrones

```
fun elimina1 :: "nat ⇒ 'a list ⇒ 'a list" where
  elimina1_Nil:  "elimina1 n [] = []"
| elimina1_Cons: "elimina1 n (x#xs) = (case n of
                                         0 => x#xs
                                       | Suc(m) => elimina1 m xs)"
```

- Dos ecuaciones en vez de tres: el segundo argumento se distingue por patrón, y el primero dentro de la ecuación, con `case`.
- La llamada recursiva está dentro del `case`: solo ocurre cuando `n = Suc m`.

Calcula lo mismo que `elimina`: `value "elimina1 2 [1::nat,2,3,4]"` ⇝ `[3, 4]`

La inducción de elimina1

```
thm elimina1.induct
```

```
⟦⋀n. P n [];
 ⋀n x xs. (⋀nat. n = Suc nat ⟹ P nat xs) ⟹ P n (x # xs)⟧
⟹ P n xs
```

La inducción de `elimina1`

```
thm elimina1.induct
```

```
⟦⋀n. P n [];
 ⋀n x xs. (⋀nat. n = Suc nat ⟹ P nat xs) ⟹ P n (x # xs)⟧
⟹ P n xs
```

- Dos casos, uno por ecuación: uno menos que `elimina.induct`.

La inducción de `elimina1`

```
thm elimina1.induct
```

```
⟦⋀n. P n [];
 ⋀n x xs. (⋀nat. n = Suc nat ⟹ P nat xs) ⟹ P n (x # xs)⟧
⟹ P n xs
```

- Dos casos, uno por ecuación: uno menos que `elimina.induct`.
- La hipótesis de inducción es condicional: vale solo si `n = Suc nat`, que es el único caso en que hay llamada recursiva.

La inducción de `elimina1`

```
thm elimina1.induct
```

```
⟦⋀n. P n [];
 ⋀n x xs. (⋀nat. n = Suc nat ⟹ P nat xs) ⟹ P n (x # xs)⟧
⟹ P n xs
```

- Dos casos, uno por ecuación: uno menos que `elimina.induct`.
- La hipótesis de inducción es condicional: vale solo si `n = Suc nat`, que es el único caso en que hay llamada recursiva.
- `nat` es el nombre que Isabelle le da a la variable ligada, no el tipo.

La inducción de `elimina1`

```
thm elimina1.induct
```

```
⟦⋀n. P n [];
 ⋀n x xs. (⋀nat. n = Suc nat ⟹ P nat xs) ⟹ P n (x # xs)⟧
⟹ P n xs
```

- Dos casos, uno por ecuación: uno menos que `elimina.induct`.
- La hipótesis de inducción es condicional: vale solo si `n = Suc nat`, que es el único caso en que hay llamada recursiva.
- `nat` es el nombre que Isabelle le da a la variable ligada, no el tipo.

Ejercicio C9 de la relación: demuestre que `elimina1 n xs = elimina n xs`.

Lo que se toma más lo que se quita, es todo

```
lemma "conc (coge n xs) (elimina n xs) = xs"
proof (induct rule: coge.induct)
  fix n
  show "conc (coge n []) (elimina n []) = []" by simp
next
  fix x xs
  show "conc (coge 0 (x#xs)) (elimina 0 (x#xs)) = x#xs" by simp
next
  fix n x xs
  assume HI: "conc (coge n xs) (elimina n xs) = xs"
  have "conc (coge (Suc n) (x#xs)) (elimina (Suc n) (x#xs)) =
        conc (x#(coge n xs)) (elimina n xs)" by simp
  also have "... = x#(conc (coge n xs) (elimina n xs))" by simp
  also have "... = x#xs" using HI by simp
  finally show "conc (coge (Suc n) (x#xs)) (elimina (Suc n) (x#xs)) = x#xs" by simp
qed
```

Y la automática

```
lemma "conc (coge n xs) (elimina n xs) = xs"
  by (induct rule: coge.induct) auto
```

Y la automática

```
lemma "conc (coge n xs) (elimina n xs) = xs"
  by (induct rule: coge.induct) auto
```

- `induct rule: coge.induct` usa la inducción de la función, no la del tipo.

Y la automática

```
lemma "conc (coge n xs) (elimina n xs) = xs"
  by (induct rule: coge.induct) auto
```

- `induct rule: coge.induct` usa la inducción de la función, no la del tipo.
- Los tres casos que abre son las tres ecuaciones de `coge`, y `auto` los cierra.

Y la automática

```
lemma "conc (coge n xs) (elimina n xs) = xs"
  by (induct rule: coge.induct) auto
```

- `induct rule: coge.induct` usa la inducción de la función, no la del tipo.
- Los tres casos que abre son las tres ecuaciones de `coge`, y `auto` los cierra.

Regla práctica: si la función tiene más de dos ecuaciones, pruebe con `f.induct`.

“Todos cumplen p”

```
fun todos :: "('a ⇒ bool) ⇒ 'a list ⇒ bool" where
  "todos p []     = True"
| "todos p (y#ys) = ((p y) ∧ (todos p ys))"
```

“Todos cumplen p”

```
fun todos :: "('a ⇒ bool) ⇒ 'a list ⇒ bool" where
  "todos p []     = True"
| "todos p (y#ys) = ((p y) ∧ (todos p ys))"
```

- El primer argumento es una función: `todos` es de orden superior.

“Todos cumplen p”

```
fun todos :: "('a ⇒ bool) ⇒ 'a list ⇒ bool" where
  "todos p []     = True"
| "todos p (y#ys) = ((p y) ∧ (todos p ys))"
```

- El primer argumento es una función: `todos` es de orden superior.
- La lista vacía da `True`: no hay ningún elemento que incumpla.

"Todos cumplen p"

```
fun todos :: "('a ⇒ bool) ⇒ 'a list ⇒ bool" where
  "todos p []     = True"
| "todos p (y#ys) = ((p y) ∧ (todos p ys))"
```

- El primer argumento es una función: `todos` es de orden superior.
- La lista vacía da `True`: no hay ningún elemento que incumpla.

```
lemma "(todos (λx. 0 < x) [2::nat,1,4]) = True" by auto
lemma "todos (λx. 1 < (longitud x)) [[2,1,4],[1,3]] = True" by simp
```

"Todos cumplen p"

```
fun todos :: "('a ⇒ bool) ⇒ 'a list ⇒ bool" where
  "todos p []     = True"
| "todos p (y#ys) = ((p y) ∧ (todos p ys))"
```

- El primer argumento es una función: `todos` es de orden superior.
- La lista vacía da `True`: no hay ningún elemento que incumpla.

```
lemma "(todos (λx. 0 < x) [2::nat,1,4]) = True" by auto
lemma "todos (λx. 1 < (longitud x)) [[2,1,4],[1,3]] = True" by simp
```

En el segundo, los elementos de la lista son listas, y la propiedad habla de su longitud.

`todos` distribuye sobre la conjunción

```
lemma "todos (λx. P x ∧ Q x) xs = (todos P xs ∧ todos Q xs)"
proof (induct xs)
  show "todos (λx. P x ∧ Q x) [] = (todos P [] ∧ todos Q [])" by simp
next
  fix a xs
  assume HI: "todos (λx. P x ∧ Q x) xs = (todos P xs ∧ todos Q xs)"
  show "todos (λx. P x ∧ Q x) (a#xs) = (todos P (a#xs) ∧ todos Q (a#xs))"
  proof -
    have "todos (λx. P x ∧ Q x) (a#xs) =
          ((P a) ∧ (Q a) ∧ todos (λx. P x ∧ Q x) xs)" by simp
    also have "... = ((P a) ∧ (Q a) ∧ todos P xs ∧ todos Q xs)" using HI by simp
    also have "... = (((P a) ∧ todos P xs) ∧ ((Q a) ∧ todos Q xs))" by auto
    also have "... = (todos P (a#xs) ∧ todos Q (a#xs))" by simp
    finally show ?thesis .
  qed
qed
```

El paso que no es de listas

- El primero, el segundo y el cuarto paso usan las ecuaciones de `todos` y la hipótesis de inducción.

El paso que no es de listas

- El primero, el segundo y el cuarto paso usan las ecuaciones de `todos` y la hipótesis de inducción.
- El tercero es pura lógica proposicional: conmutar y asociar conjunciones. Por eso ahí va `auto` y no `simp`.

El paso que no es de listas

- El primero, el segundo y el cuarto paso usan las ecuaciones de `todos` y la hipótesis de inducción.
- El tercero es pura lógica proposicional: conmutar y asociar conjunciones. Por eso ahí va `auto` y no `simp`.

Las reglas de la Parte I siguen ahí, trabajando por debajo de `auto`.

Y la versión corta

```
lemma "todos (λx. P x ∧ Q x) xs = (todos P xs ∧ todos Q xs)"
proof (induct xs)
  show "todos (λx. P x ∧ Q x) [] = (todos P [] ∧ todos Q [])" by simp
next
  fix a xs
  assume "todos (λx. P x ∧ Q x) xs = (todos P xs ∧ todos Q xs)"
  thus "todos (λx. P x ∧ Q x) (a#xs) = (todos P (a#xs) ∧ todos Q (a#xs))"
    by auto
qed
```

Y la versión corta

```
lemma "todos (λx. P x ∧ Q x) xs = (todos P xs ∧ todos Q xs)"
proof (induct xs)
  show "todos (λx. P x ∧ Q x) [] = (todos P [] ∧ todos Q [])" by simp
next
  fix a xs
  assume "todos (λx. P x ∧ Q x) xs = (todos P xs ∧ todos Q xs)"
  thus "todos (λx. P x ∧ Q x) (a#xs) = (todos P (a#xs) ∧ todos Q (a#xs))"
    by auto
qed

lemma "todos (λx. P x ∧ Q x) xs = (todos P xs ∧ todos Q xs)"
  by (induct xs) auto
```

Y la versión corta

```
lemma "todos (λx. P x ∧ Q x) xs = (todos P xs ∧ todos Q xs)"
proof (induct xs)
  show "todos (λx. P x ∧ Q x) [] = (todos P [] ∧ todos Q [])" by simp
next
  fix a xs
  assume "todos (λx. P x ∧ Q x) xs = (todos P xs ∧ todos Q xs)"
  thus "todos (λx. P x ∧ Q x) (a#xs) = (todos P (a#xs) ∧ todos Q (a#xs))"
    by auto
qed

lemma "todos (λx. P x ∧ Q x) xs = (todos P xs ∧ todos Q xs)"
  by (induct xs) auto
```

`thus` es `from this show`: encadena la hipótesis recién supuesta con lo que sigue.

Cierre

Lo que hay que llevarse

- Un tipo recursivo da a la vez la forma de las definiciones y la forma de las demostraciones.

Lo que hay que llevarse

- Un tipo recursivo da a la vez la forma de las definiciones y la forma de las demostraciones.
- Se induce sobre el argumento en el que recursa la función; si son dos, se usa `f.induct`.

Lo que hay que llevarse

- Un tipo recursivo da a la vez la forma de las definiciones y la forma de las demostraciones.
- Se induce sobre el argumento en el que recursa la función; si son dos, se usa `f.induct`.
- Lo que está en la definición sale por `simp`; lo que no, hay que demostrarlo.

Lo que hay que llevarse

- Un tipo recursivo da a la vez la forma de las definiciones y la forma de las demostraciones.
- Se induce sobre el argumento en el que recursa la función; si son dos, se usa `f.induct`.
- Lo que está en la definición sale por `simp`; lo que no, hay que demostrarlo.
- `quickcheck` antes de demostrar; `oops` cuando resulta falso.

Lo que hay que llevarse

- Un tipo recursivo da a la vez la forma de las definiciones y la forma de las demostraciones.
- Se induce sobre el argumento en el que recursa la función; si son dos, se usa `f.induct`.
- Lo que está en la definición sale por `simp`; lo que no, hay que demostrarlo.
- `quickcheck` antes de demostrar; `oops` cuando resulta falso.
- `fun` calcula, `definition` describe. `value` solo puede con la primera.

Para trabajar

La relación `Ejercicios_programacion_funcional.thy`: una parte por teoría.

Para trabajar

La relación `Ejercicios_programacion_funcional.thy`: una parte por teoría.

- A — listas: `inversa`, `conc`, `longitud`, y la inversa con acumulador.

Para trabajar

La relación `Ejercicios_programacion_funcional.thy`: una parte por teoría.

- A — listas: `inversa`, `conc`, `longitud`, y la inversa con acumulador.
- B — naturales: sumas, `par` e `impar` con existenciales, `par1` frente a `par`.

Para trabajar

La relación `Ejercicios_programacion_funcional.thy`: una parte por teoría.

- A — listas: `inversa`, `conc`, `longitud`, y la inversa con acumulador.
- B — naturales: sumas, `par` e `impar` con existenciales, `par1` frente a `par`.
- C — `repite`, `coge`, `elimina`, `todos`; y definir `algun`.

Para trabajar

La relación `Ejercicios_programacion_funcional.thy`: una parte por teoría.

- A — listas: `inversa`, `conc`, `longitud`, y la inversa con acumulador.
- B — naturales: sumas, `par` e `impar` con existenciales, `par1` frente a `par`.
- C — `repite`, `coge`, `elimina`, `todos`; y definir `algun`.

Además de demostrar, hay que definir funciones, refutar con `quickcheck` y demostrar sin inducción.

Para trabajar

La relación `Ejercicios_programacion_funcional.thy`: una parte por teoría.

- A — listas: `inversa`, `conc`, `longitud`, y la inversa con acumulador.
- B — naturales: sumas, `par` e `impar` con existenciales, `par1` frente a `par`.
- C — `repite`, `coge`, `elimina`, `todos`; y definir `algun`.

Además de demostrar, hay que definir funciones, refutar con `quickcheck` y demostrar sin inducción.

Dificultad marcada de [*] a [***]. Antes de demostrar, pruebe el enunciado con `value` o `quickcheck`.